

Documentation

Introduction :

Ce projet Java utilise Maven pour la gestion des dépendances et met en œuvre JPA avec Hibernate pour l'interaction avec une base de données PostgreSQL. Il se concentre sur la modélisation des magasins d'animaux et des produits, offrant des fonctionnalités de gestion et de manipulation de données via des services dédiés.

Structure du Projet :

1.Dépendances

- Hibernate Core 6.4.0.Final: Intégration de la couche de persistance.
- Hibernate JCache 6.2.5.Final: Support pour la gestion du cache.
- Ehcache 3.10.8: Gestion du cache.
- JAXB Runtime 4.0.3: Pour la sérialisation/désérialisation XML.
- PostgreSQL 42.7.1: Base de données utilisée.
- JUnit Jupiter API 5.8.2: Pour les tests unitaires.

2.Packages :

- Enums: FishLivEv, ProdType.
- Exceptions: PetStoreException, AddressException, ProductException.
- Entités: Address, Animal, Cat, Fish, PetStore, Product.

3.Services :

AbstractService

La classe `AbstractService` sert de base pour les services. Elle gère la configuration de l'EntityManagerFactory et de l'EntityManager pour simplifier l'accès aux opérations de persistance.

AddressService

Le `AddressService` gère la gestion des adresses. Voici un aperçu des méthodes offertes :

- createAddress: Crée une nouvelle adresse dans la base de données.
- getAllAddresses: Récupère toutes les adresses enregistrées.
- findAddressById: Recherche une adresse spécifique par son ID.
- updateAddress: Met à jour une adresse existante.
- deleteAddress: Supprime une adresse par son ID.

PetStoreService

Le `PetStoreService` s'occupe des opérations liées aux magasins d'animaux. Voici un résumé des

4. méthodes :

- createPetStore: Crée un nouveau magasin d'animaux dans la base de données
- getAllPetStores: Récupère tous les magasins d'animaux enregistrés dans la base de données.
- findPetStoreById: Recherche un magasin d'animaux par son ID.
- updatePetStore: Met à jour un magasin d'animaux existant avec les informations du magasin mis à jour.
- deletePetStore: Supprime un magasin d'animaux par son ID.

ProductService :

Le `ProductService` gère les opérations CRUD pour les produits. Voici un aperçu des fonctionnalités :

- createProduct: Crée un nouveau produit dans la base de données.
- getAllProduct: Récupère tous les produits enregistrés dans la base de données.
- findProductById: Recherche un produit par son ID.
- updateProduct: Met à jour un produit existant avec les informations du produit mis à jour.
- deleteProduct: Supprime un produit par son ID.

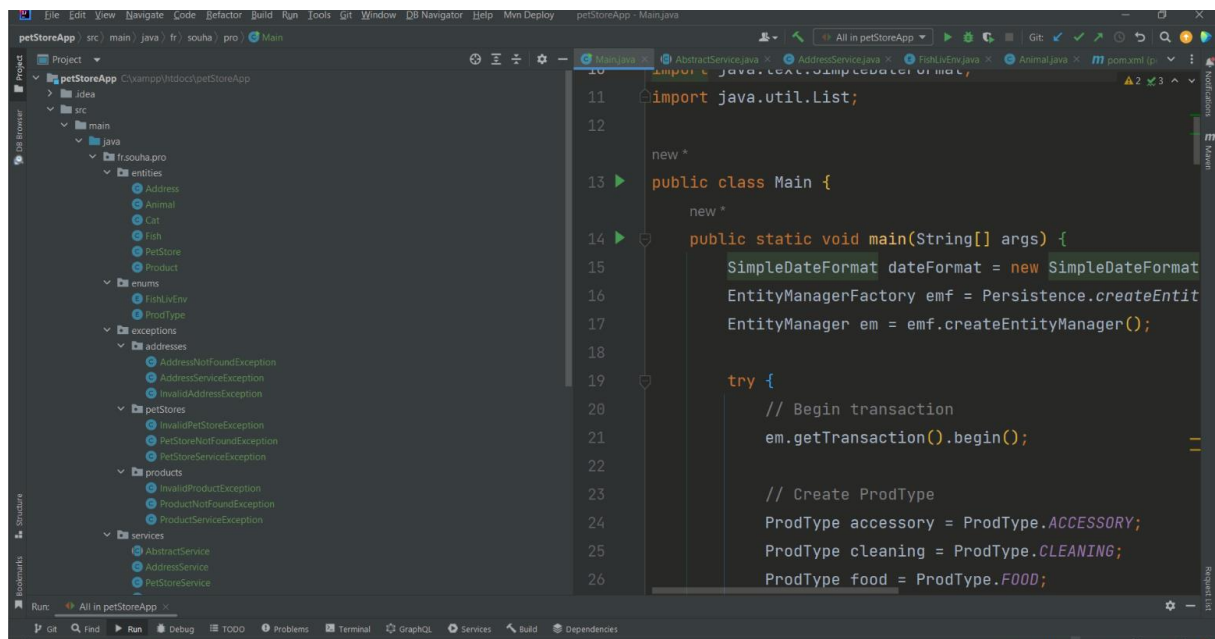


Figure 1 structure projet

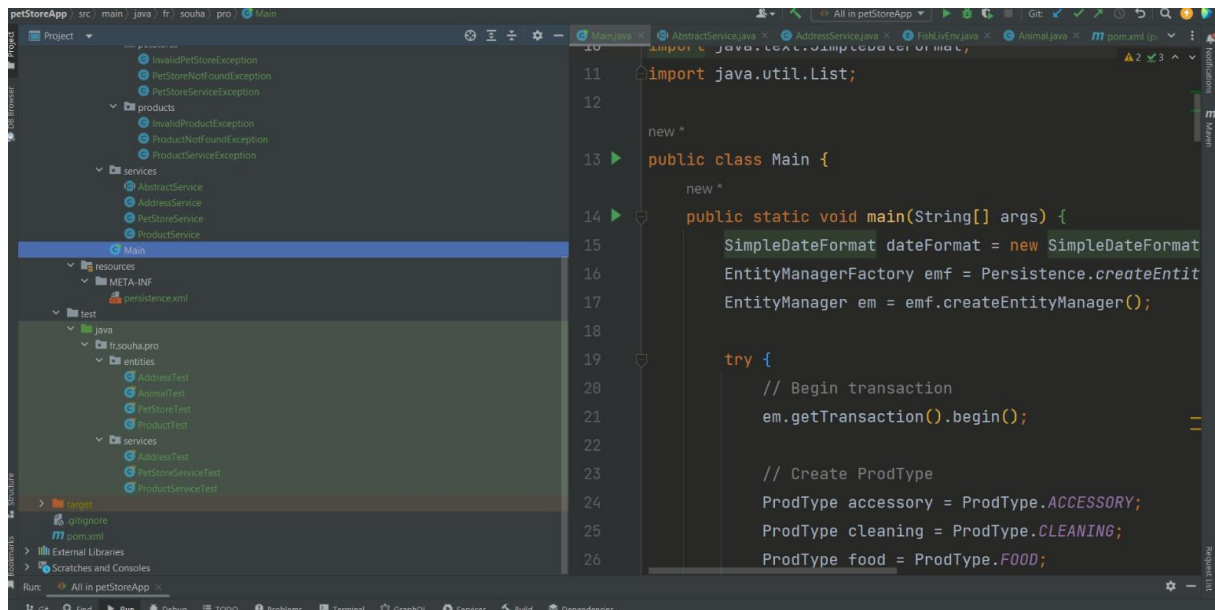


Figure 2 Structure de projet

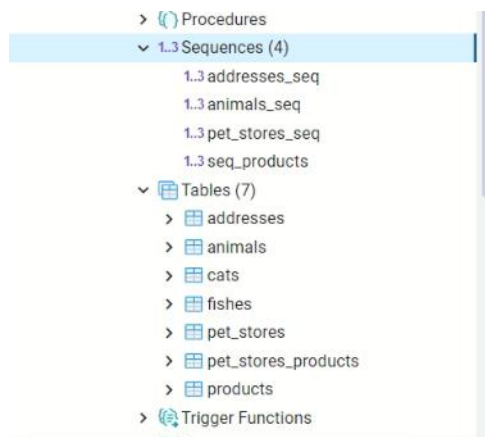
Configuration de la Base de Données :

1 Base de Données

Nom de la Base de Données: petstore

Persistence Unit: petstore

Le schéma donnes :



Le diagramme de ma BaseDonnes :

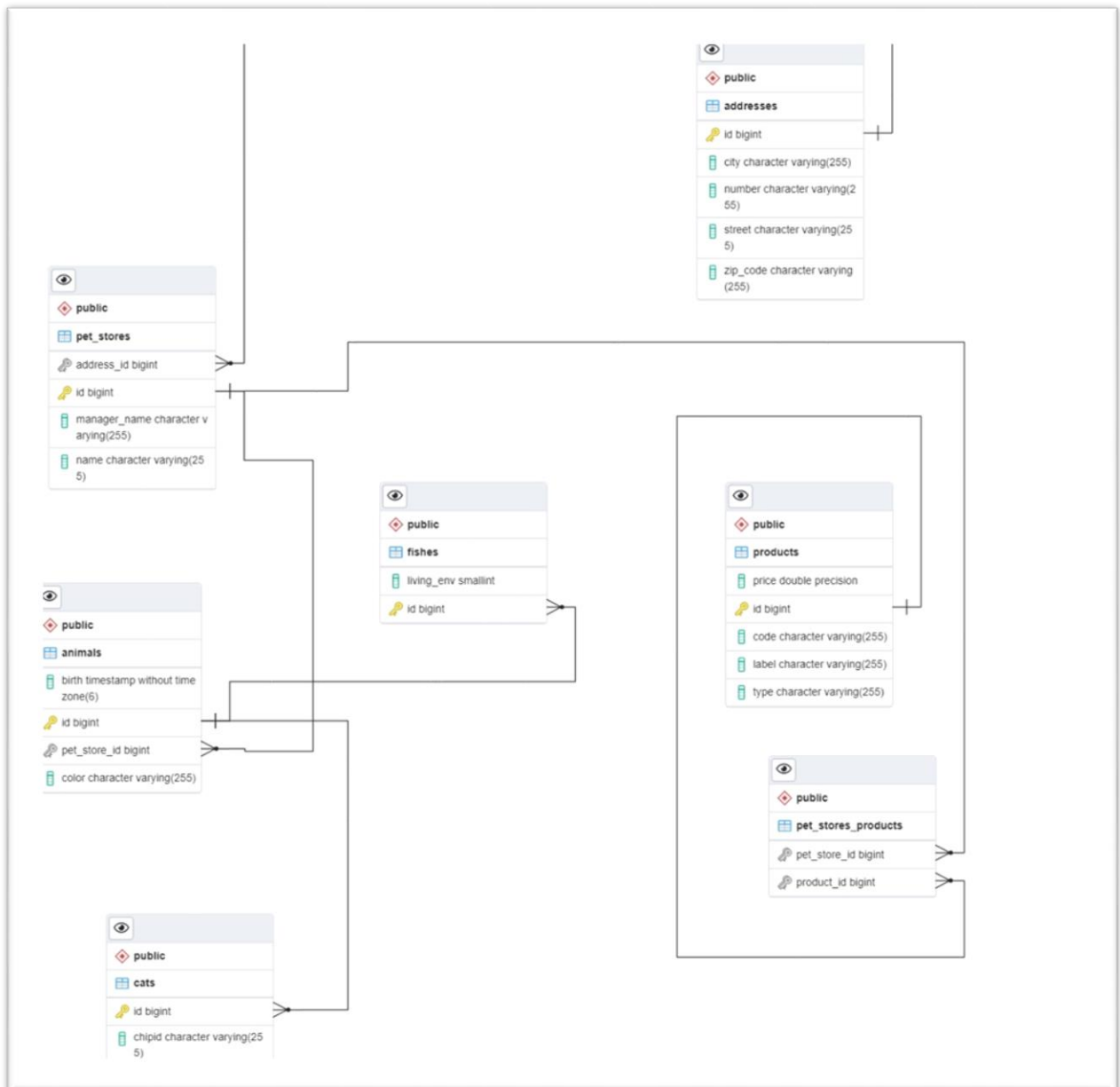


Figure 3 Diagramme de BD

2 Fichier persistence.xml

Le fichier `persistence.xml` situé dans `src/main/resources/META-INF/` contient la configuration de la persistence-unit. Il est configuré les propriétés de connexion de la base de données.

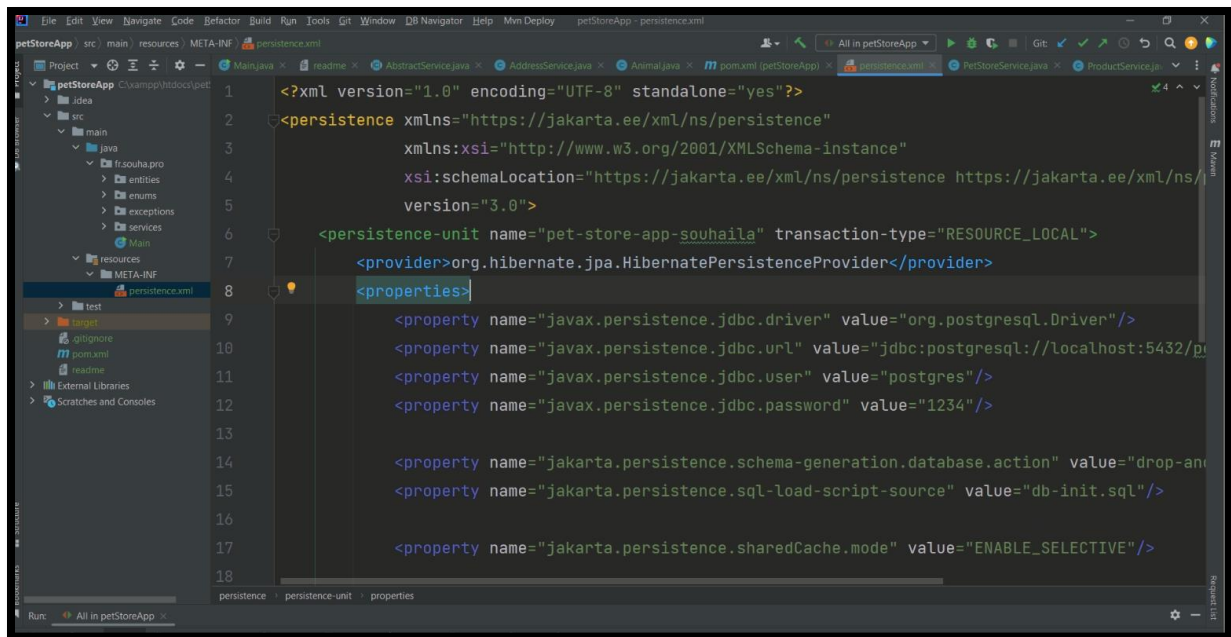


Figure 4 persistence.xml

Mapping des Entités JPA :

Entités:

J'ai créé les classes d'entités JPA correspondant à mon modèle (Animal, Fish, Cat, etc.).

J'ai ajouté les annotations pour mapper les champs aux colonnes de la base de données.

Relations:

- Relation One-to-Many: Entre PetStore et Animals.
- Relation One-to-One: Entre Address et PetStore.
- Relation Many-to-Many: Entre PetStores et Produits.

Héritage:

J'ai utilisé l'annotation `@Inheritance` avec la stratégie "1 table par classe" (`@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`).

- Relation d'héritage: Animal et Cat, Animal et Fish.

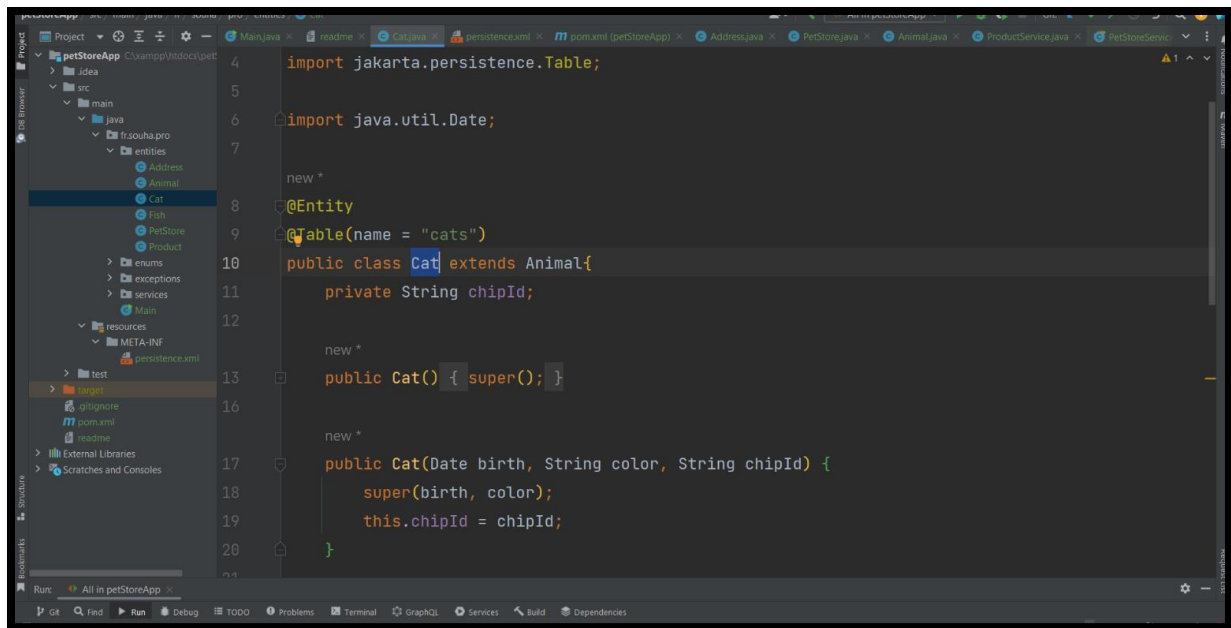


Figure 5 Relation héritage Animal Cat

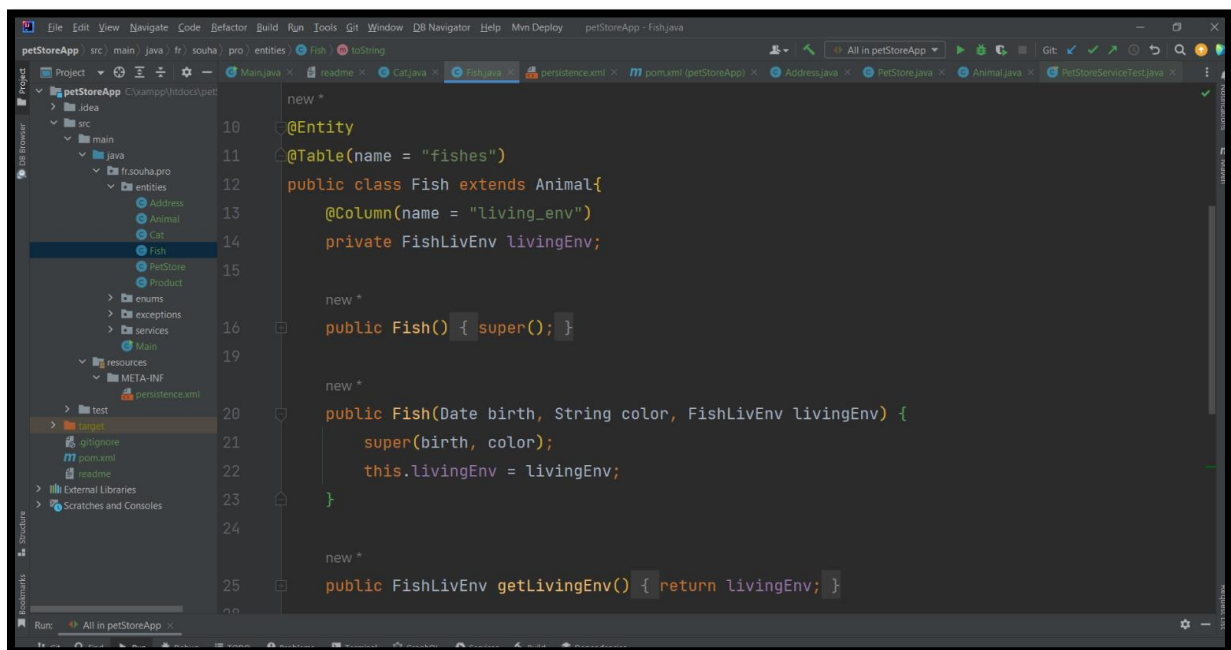


Figure 6 Relation héritage Animal Fish

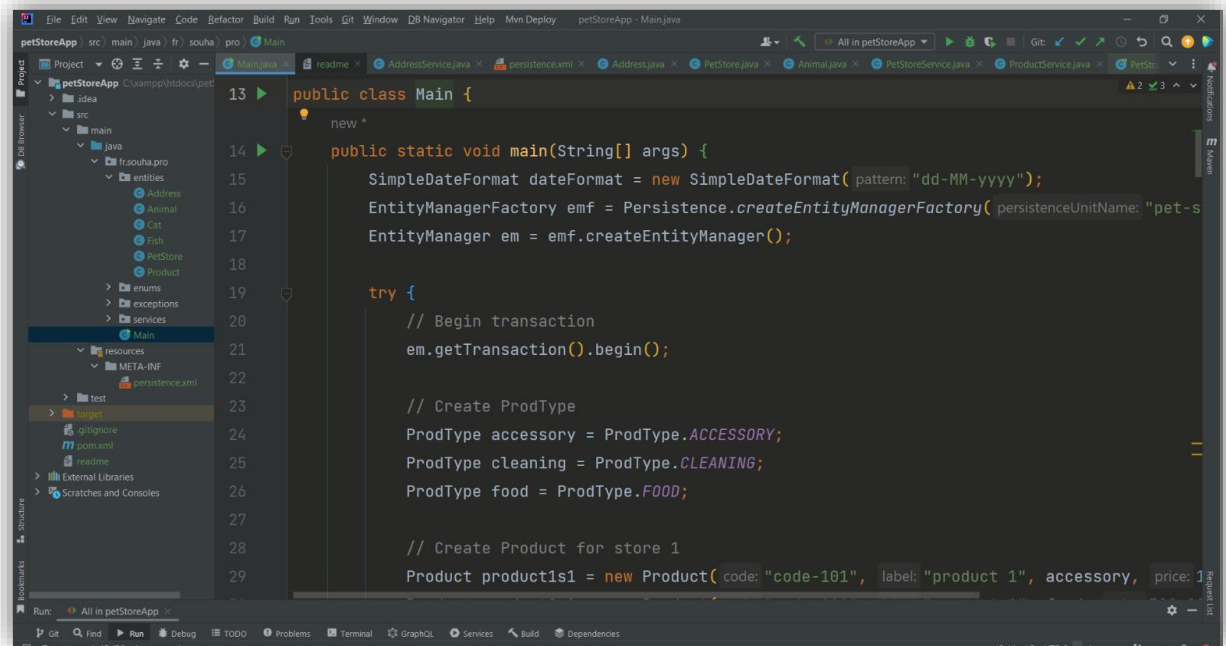


Figure 7 Réalisation des tests sur la classe Main

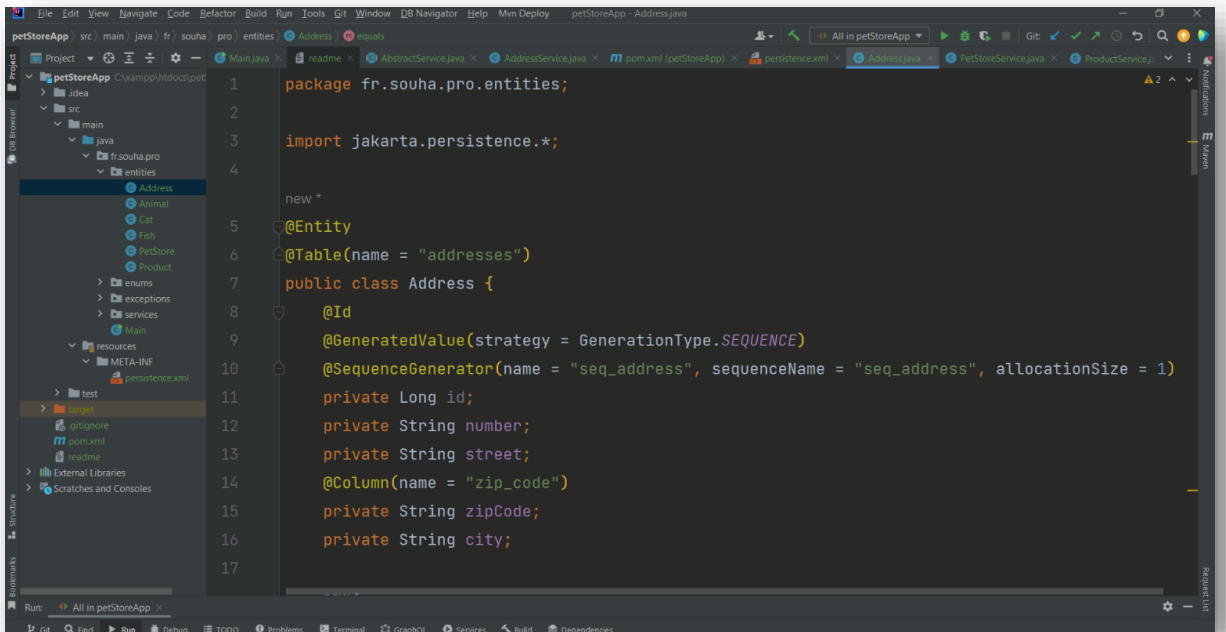


Figure 8 Entity Adresse

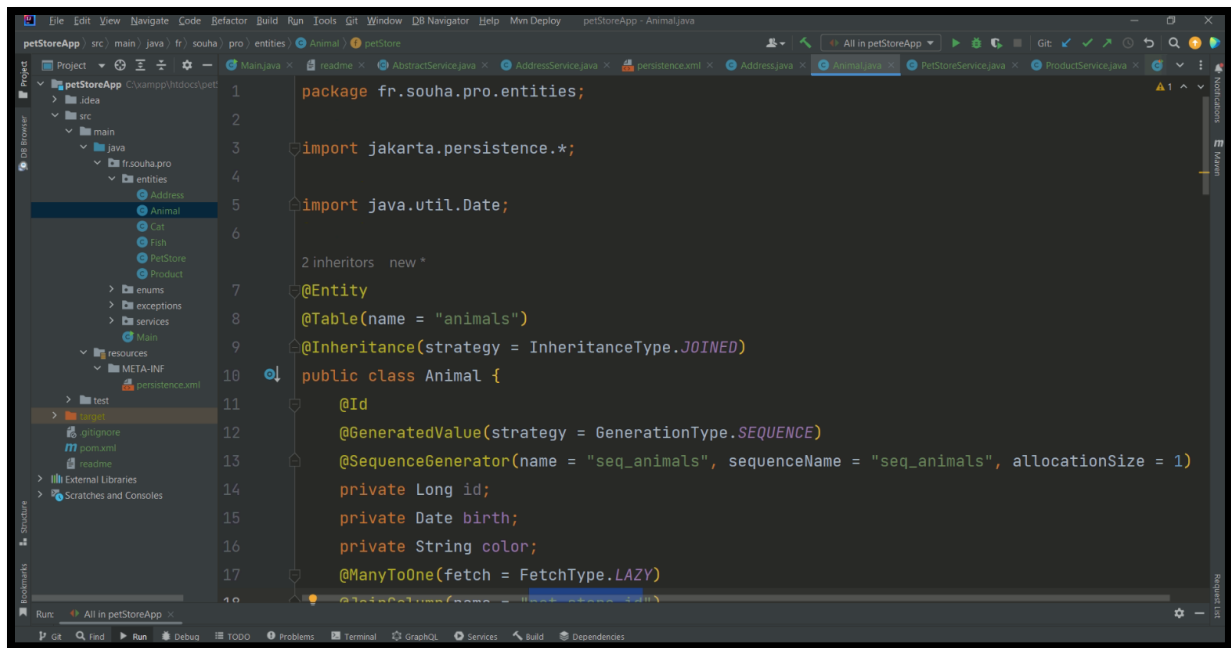


Figure 9 Génération de séquence d'ids

Exécution du Projet :

Pour exécuter le projet, suivez ces étapes :

Configurez la base de données dans `persistence.xml`.

Utilisez Maven pour compiler et exécuter l'application.

```

mvn clean install
mvn exec:java

```

Les exceptions:

Les exceptions dans ce projet sont conçues pour gérer efficacement les erreurs et les cas exceptionnels lors des opérations liées aux entités, aux services et à la manipulation des données dans la base de données.

PetStoreException

Description : Cette exception est utilisée pour signaler des problèmes spécifiques liés aux opérations effectuées dans la couche des magasins d'animaux (PetStore). Elle couvre une gamme variée de situations, de l'absence d'un magasin spécifique à des erreurs lors de la création, de la mise à jour ou de la suppression des magasins.

AddressException

Description : L'exception AddressException est dédiée à la gestion des erreurs spécifiques aux opérations relatives aux adresses. Elle est utilisée pour signaler des problèmes lors de la création, de la recherche, de la mise à jour ou de la suppression d'adresses associées aux entités.

ProductException

Description : Cette exception concerne les erreurs rencontrées lors de la manipulation des produits au sein de l'application. Elle couvre des cas tels que la non-existence d'un produit spécifique, des problèmes lors de la création, de la recherche, de la mise à jour ou de la suppression de produits.

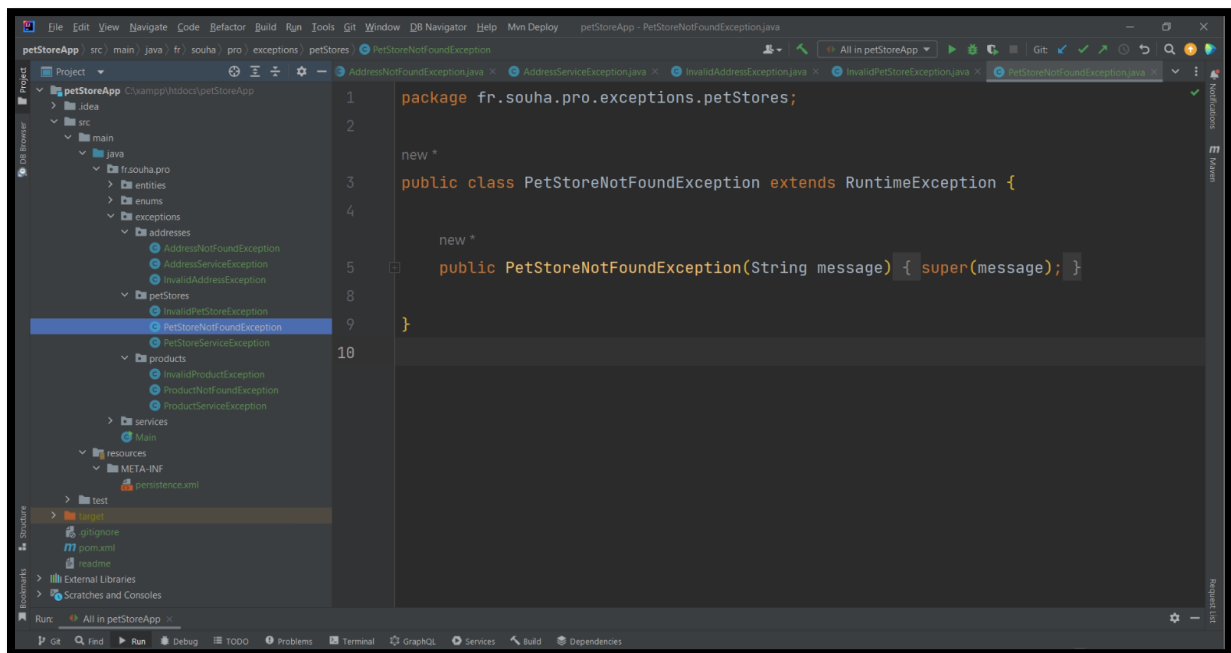


Figure 10Exception de petStores

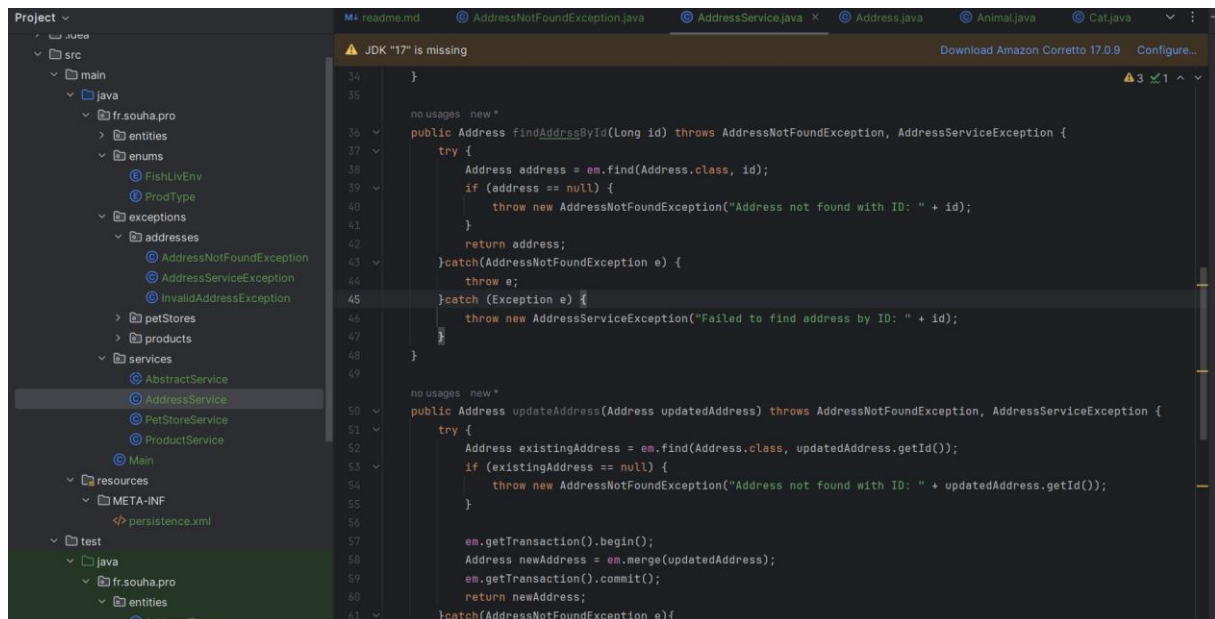


Figure 11 Consommation d'exception

Les tests unitaires:

Les tests unitaires sont une partie cruciale du processus de développement, garantissant que chaque composant du projet fonctionne comme prévu de manière isolée. Pour le projet PetStore, j'ai mis en place des tests unitaires couvrant divers aspects du code, y compris les entités JPA, les services, et l'accès à la base de données.

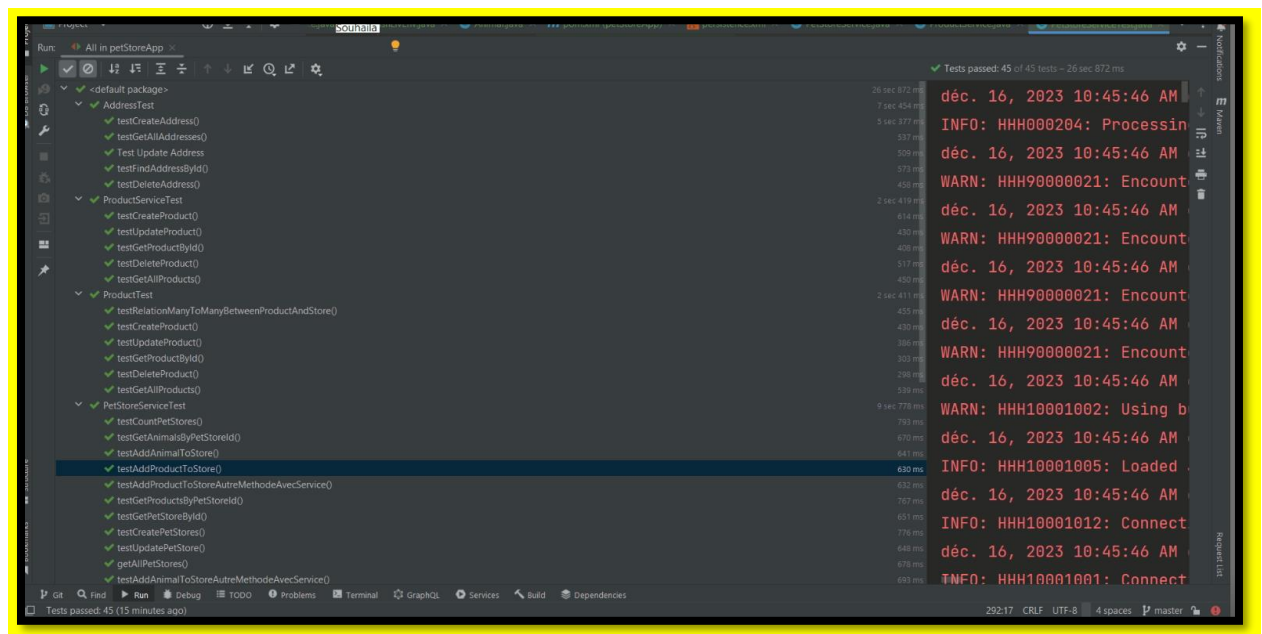


Figure 12 Les résultats des tests unitaires de mon projet

